

O&G · 1 — Decline curve analysis with Arps equations ⇔

The most common forecasting tool in production engineering is the **Arps decline curve**. Rather than treat exponential, harmonic, and hyperbolic decline as three separate formulas, we write them as a single ODE and let the *decline exponent* b interpolate between them — exactly the kind of unification SciML's solver stack makes easy.

The nominal decline rate is defined as $D(t) = -\frac{1}{q} \frac{dq}{dt}$. Arps assumes $D(t) = \frac{D_i}{1 + bD_it}$, which is equivalent to the autonomous ODE

$$\frac{dq}{dt} = -D_i \frac{q^{1+b}}{q_i^b}, \quad q(0) = q_i$$

- $b = 0$
→ exponential decline
- $b = 1$
→ harmonic decline
- $0 < b < 1$
→ hyperbolic decline (the usual case for unconventional wells)

```
1 using OrdinaryDiffEq, Optimization, OptimizationOptimJL, ForwardDiff, Plots, Random, PlutoUI
```

1. The unified Arps ODE ⇔

We track two states: the instantaneous rate q and the cumulative production $Q = \int q dt$, so a single `solve` call gives us both the forecast and the reserves estimate.

```
arps_system! (generic function with 1 method)
```

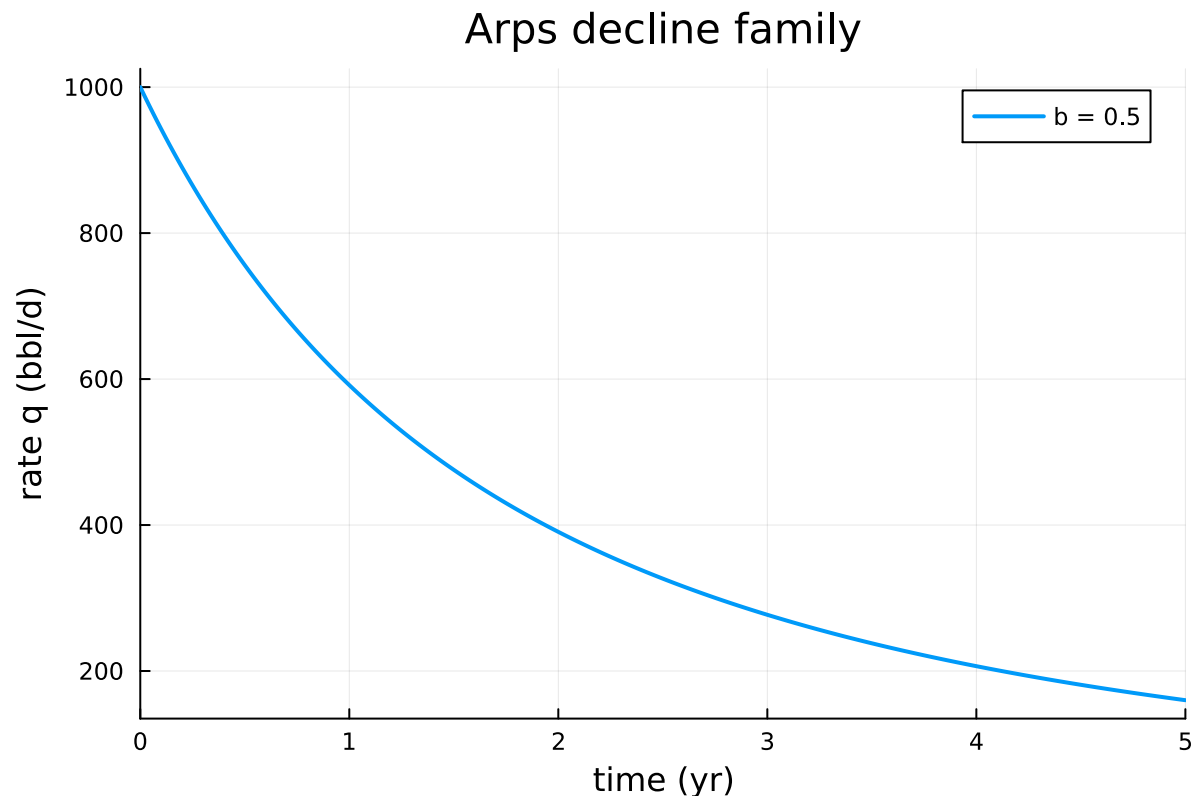
```
1 function arps_system!(du, u, p, t)
2     q, Q = u
3     qi, Di, b = p
4     du[1] = -Di * q^(1 + b) / qi^b
5     du[2] = q
6     return nothing
7 end
```

2. Compare the three decline regimes $\Leftrightarrow$

Drag b to morph continuously from exponential to harmonic decline.

 0.5

```
1 @bind b_demo Slider(0.0:0.1:1.0, default=0.5, show_value=true)
```



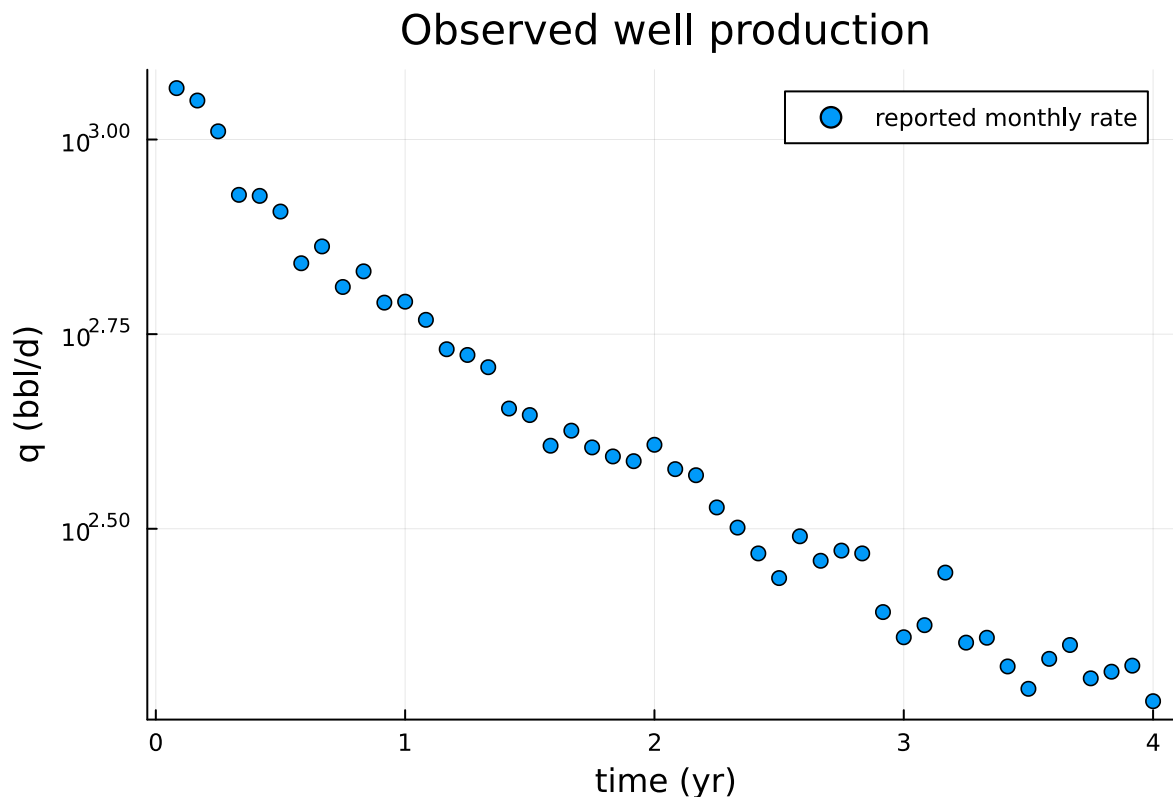
```
1 begin
2     qi_demo, Di_demo = 1000.0, 0.6 # bbl/d, 1/yr
3     demo_prob = ODEProblem(arps_system!, [qi_demo, 0.0], (0.0, 5.0), (qi_demo, Di_demo,
4     b_demo))
5     demo_sol = solve(demo_prob, Tsit5(); saveat=0.02)
6     plot(demo_sol; idxs=1, xlabel="time (yr)", ylabel="rate q (bbl/d)", label="b =
7     $(b_demo)",
8         title="Arps decline family", lw=2)
9 end
```

3. Synthetic well data ⇌

In practice we don't know (q_i, D_i, b) — we only have noisy monthly production reports and need to back them out. We simulate a "true" hyperbolic well and add realistic (multiplicative) noise.

► [1164.18, 1122.27, 1024.55, 849.038, 846.459, 808.131, 693.533, 728.941, 646.524, 677.123, 617

```
1 begin
2   rng_ogl = Random.default_rng()
3   Random.seed!(rng_ogl, 3)
4
5   true_qi, true_Di, true_b = 1200.0, 0.85, 0.6
6   t_hist = 1/12:1/12:4.0           # 4 years of monthly reports
7
8   true_prob = ODEProblem(arps_system!, [true_qi, 0.0], (0.0, 4.0), (true_qi, true_Di,
9   true_b))
10  true_sol = solve(true_prob, Tsit5(); saveat=t_hist)
11  true_rates = [u[1] for u in true_sol.u]
12
13  observed_rates = true_rates .* (1 .+ 0.06 .* randn(rng_ogl, length(true_rates)))
14 end
```



```
1 scatter(t_hist, observed_rates; label="reported monthly rate", xlabel="time (yr)",
2         ylabel="q (bbl/d)",
3         title="Observed well production", yscale=:log10)
```

4. Fit (q_i, D_i, b) to the data ⇔

Decline-curve fitting is nonlinear regression: minimize the mismatch between the model and the reported rates, in log-space (rates are best compared multiplicatively, not additively).

decline_loss (generic function with 1 method)

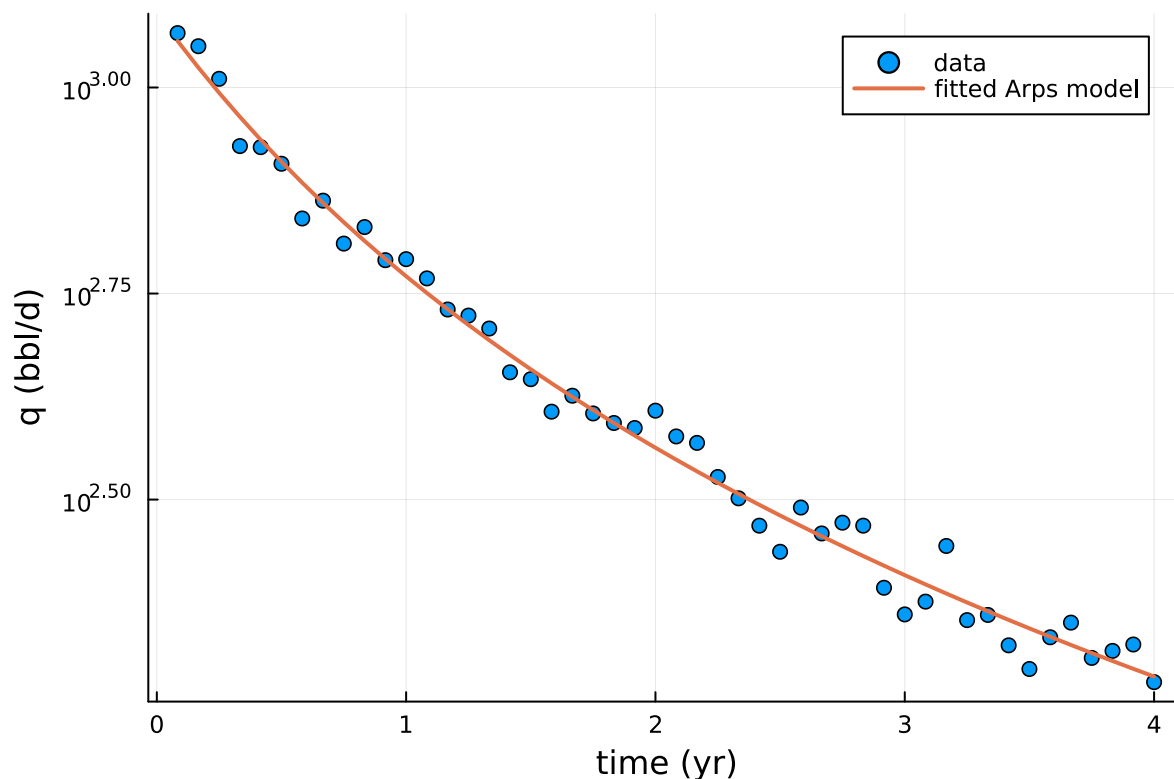
```
1 function decline_loss(logp, _)
2     qi = exp(logp[1]); Di = exp(logp[2]); b = exp(logp[3])
3     prob = remake(true_prob; p=(qi, Di, b), u0=[qi, 0.0])
4     sol = solve(prob, Tsit5(); saveat=t_hist)
5     sol.retcode == ReturnCode.Success || return Inf
6     model_rates = [u[1] for u in sol.u]
7     return sum(abs2, log.(observed_rates) .- log.(model_rates))
8 end
```

► SciMLBase.OptimizationSolution{Float64, 1, Vector{Float64}, OptimizationBase.OptimizationCache{Op

```
1 begin
2     p0_decline = log.([800.0, 0.5, 0.3]) # deliberately rough initial guess, log-
      parameterized
3     optf_decline = OptimizationFunction(decline_loss, Optimization.AutoForwardDiff())
4     optprob_decline = OptimizationProblem(optf_decline, p0_decline)
5     fit_decline = Optimization.solve(optprob_decline, OptimizationOptimJL.LBFGS())
6 end
```

► (fitted_qi = 1231.88, fitted_Di = 0.96932, fitted_b = 0.717305, true_qi = 1200.0, true_Di = 0.8!

```
1 begin
2     fitted_qi_1, fitted_Di_1, fitted_b_1 = exp.(fit_decline.u)
3     (fitted_qi=fitted_qi_1, fitted_Di=fitted_Di_1, fitted_b=fitted_b_1,
4      true_qi=true_qi, true_Di=true_Di, true_b=true_b)
5 end
```



```

1 begin
2   fitted_p_1 = exp.(fit_decline.u)
3   fitted_sol_1 = solve(remake(true_prob; p=Tuple(fitted_p_1), u0=[fitted_p_1[1],
4   0.0]), Tsit5(); saveat=t_hist)
5   scatter(t_hist, observed_rates; label="data", xlabel="time (yr)", ylabel="q
6   (bbl/d)", yscale=:log10)
7   plot!(t_hist, [u[1] for u in fitted_sol_1.u]; label="fitted Arps model", lw=2)
8 end

```

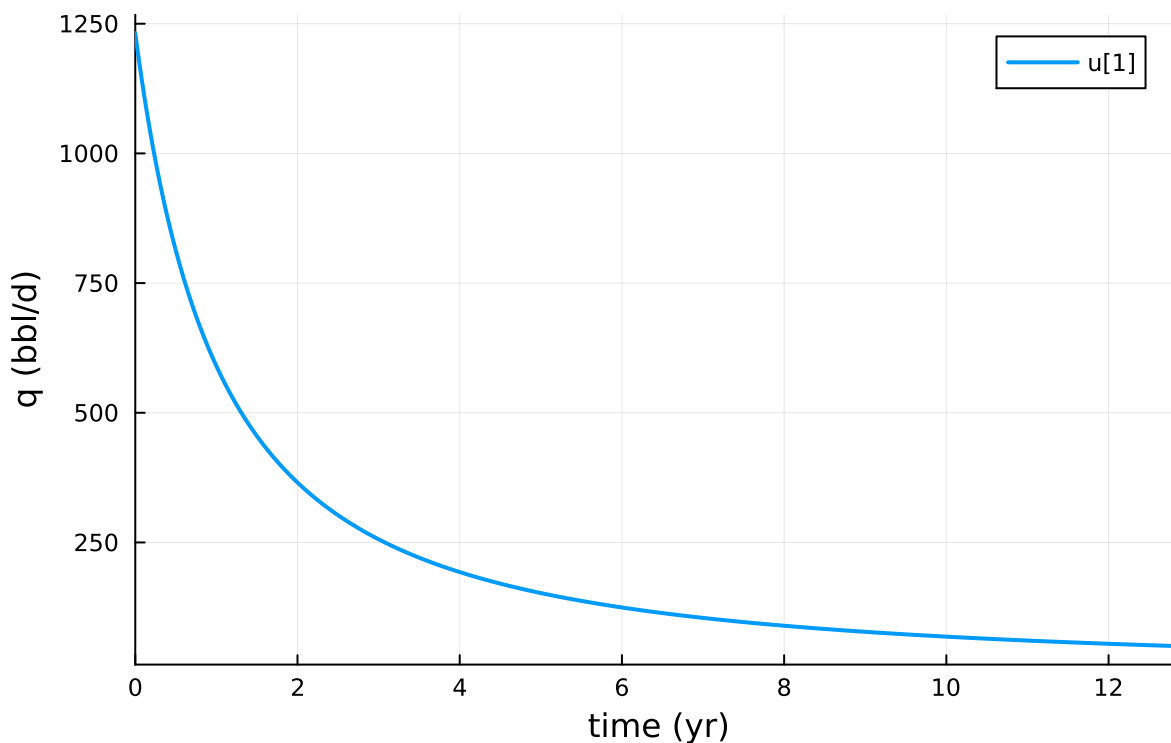
5. Forecast to the economic limit and estimate EUR ⇐

Once fitted, we forecast forward until the rate drops to an economic limit (say 50 bbl/d) using a `ContinuousCallback` to stop the integration exactly at that crossing, then read off the cumulative production — the well's **Estimated Ultimate Recovery (EUR)**.

978314.1511108304

```
1 begin
2     economic_limit = 50.0
3     condition(u, t, integrator) = u[1] - economic_limit
4     affect!(integrator) = terminate!(integrator)
5     economic_cb = ContinuousCallback(condition, affect!)
6
7     forecast_prob = remake(true_prob; p=Tuple(fitted_p_1), u0=[fitted_p_1[1], 0.0],
8                             tspan=(0.0, 100.0))
9     forecast_sol = solve(forecast_prob, Tsit5(); callback=economic_cb)
10
11     economic_life_years = forecast_sol.t[end]
12     eur_bbl = forecast_sol.u[end][2] * 365.25 # rate was per day, time in years
13 end
```

Forecast to economic limit (12.9 yr, EUR $\approx$ 978314 bb)



```
1 plot(forecast_sol; idxs=1, xlabel="time (yr)", ylabel="q (bbl/d)",
2       title="Forecast to economic limit ($(round(economic_life_years, digits=1)) yr, EUR
3       ≈ $(round(Int, eur_bbl)) bbl)", lw=2)
```

Takeaways ⇄

- Writing exponential/harmonic/hyperbolic decline as one autonomous ODE turns three textbook formulas into a single `solve` call.
- Decline-curve fitting is just nonlinear least squares on the ODE's parameters — the same recipe as `04_parameter_estimation.jl`, applied to production data instead of an epidemic curve.
- An event (ContinuousCallback) cleanly stops the forecast exactly at the economic limit, giving EUR without hand-picking a stopping time.

Next: [02_neural_decline_forecasting.jl](#) tackles wells whose production doesn't follow a clean Arps curve at all.

